SLIIT UNI
THE KNOWLEDGE UNIVERSITY

Sri Lanka Institute of Information Technology

Faculty of Computing

Prof.Sanvitha Kasthuriarachchi

IT2130 - Operating Systems and System Administration

Year 02 and Semester 02

Lecture 11

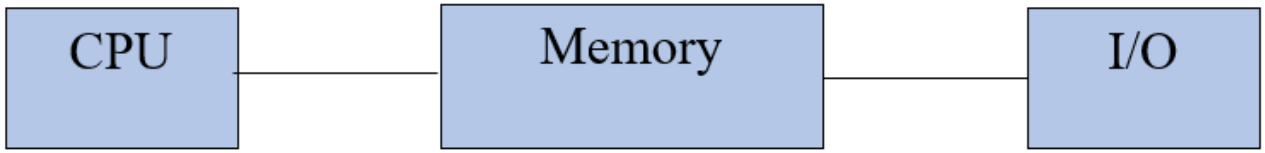
Main Memory and Virtual Memory Management in Operating System

Content

- Main memory concept
- Why main memory
- Address binding
- Contiguous Memory Allocation
- Fragmentation
- Paging

Main Memory

- Memory is a large array of words or bytes, each with its own address.
- It is a repository of quickly accessible data shared by CPU and I/O devices.
- Memory and registers are the only storage that CPU can directly access.



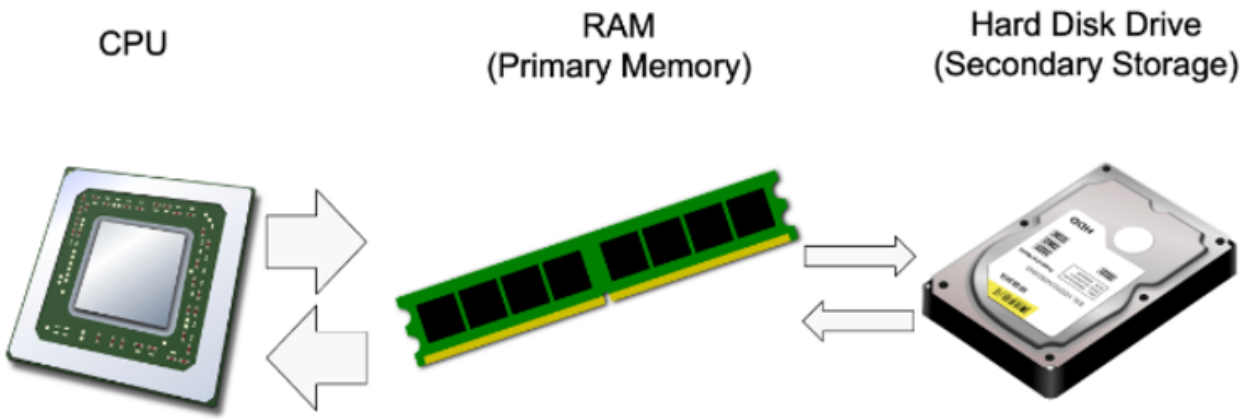
Main Memory (Cont.)

- Main memory is a **volatile** storage device. It loses its contents in the case of system failure.
- A program must be mapped to absolute addresses and loaded into memory.
- Main memory is usually divided into two partitions:
  - **Kernel memory area:** Resident operating system, PCBs, System calls usually held in low memory.
  - **User memory area:** User processes are in here. It is the high memory.

Why Main Memory

.

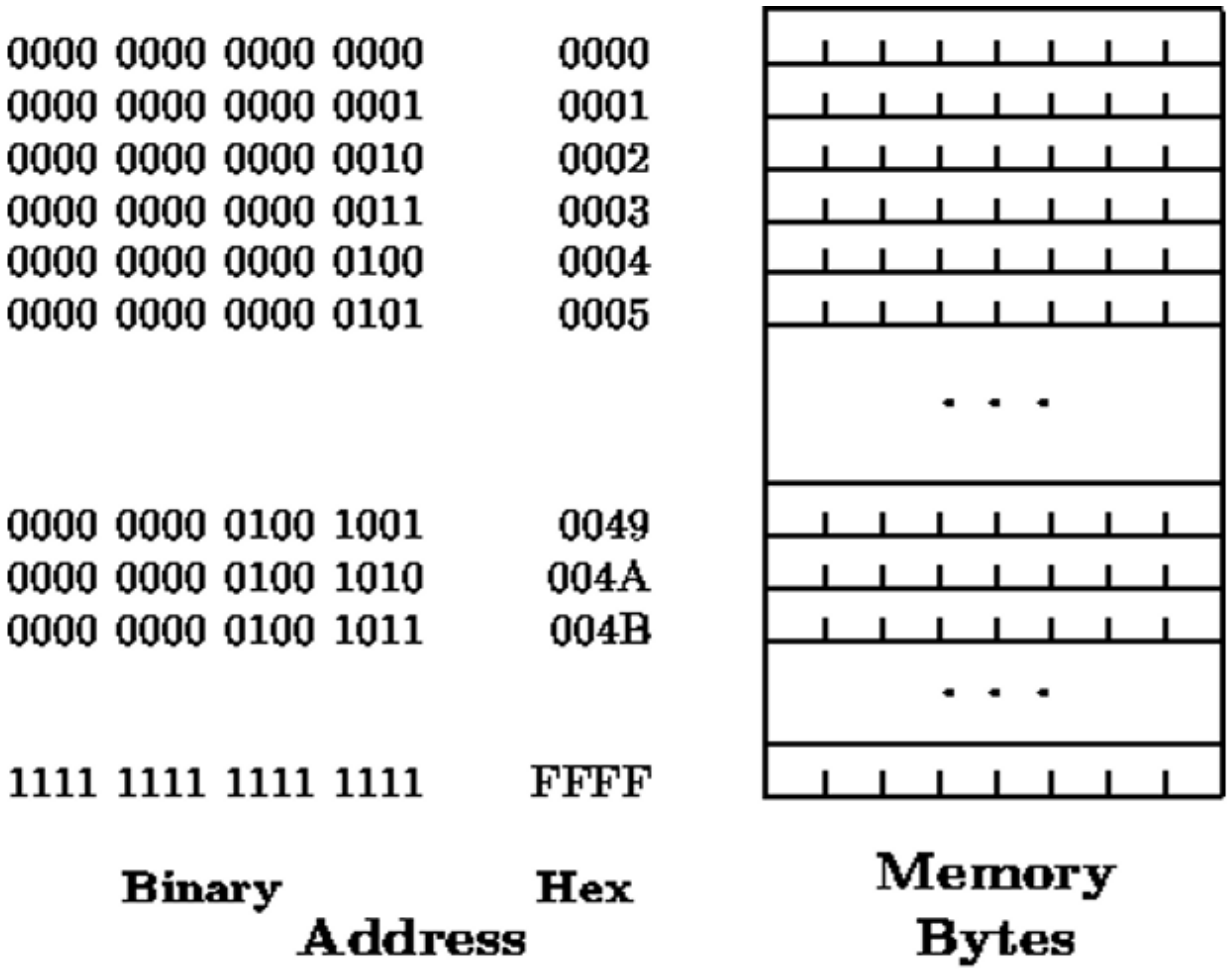
- From the hard disk the programs must be taken into the main memory prior to beginning the execution.



What OS Does for Memory Management

- The OS is responsible for the following activities:
  - Keep track of which parts of memory are being used and by whom.
  - Decide which processes to load next when memory space becomes available.
  - Allocate and deallocate memory.

For n bits address, the memory capacity $= 2^n$ bytes



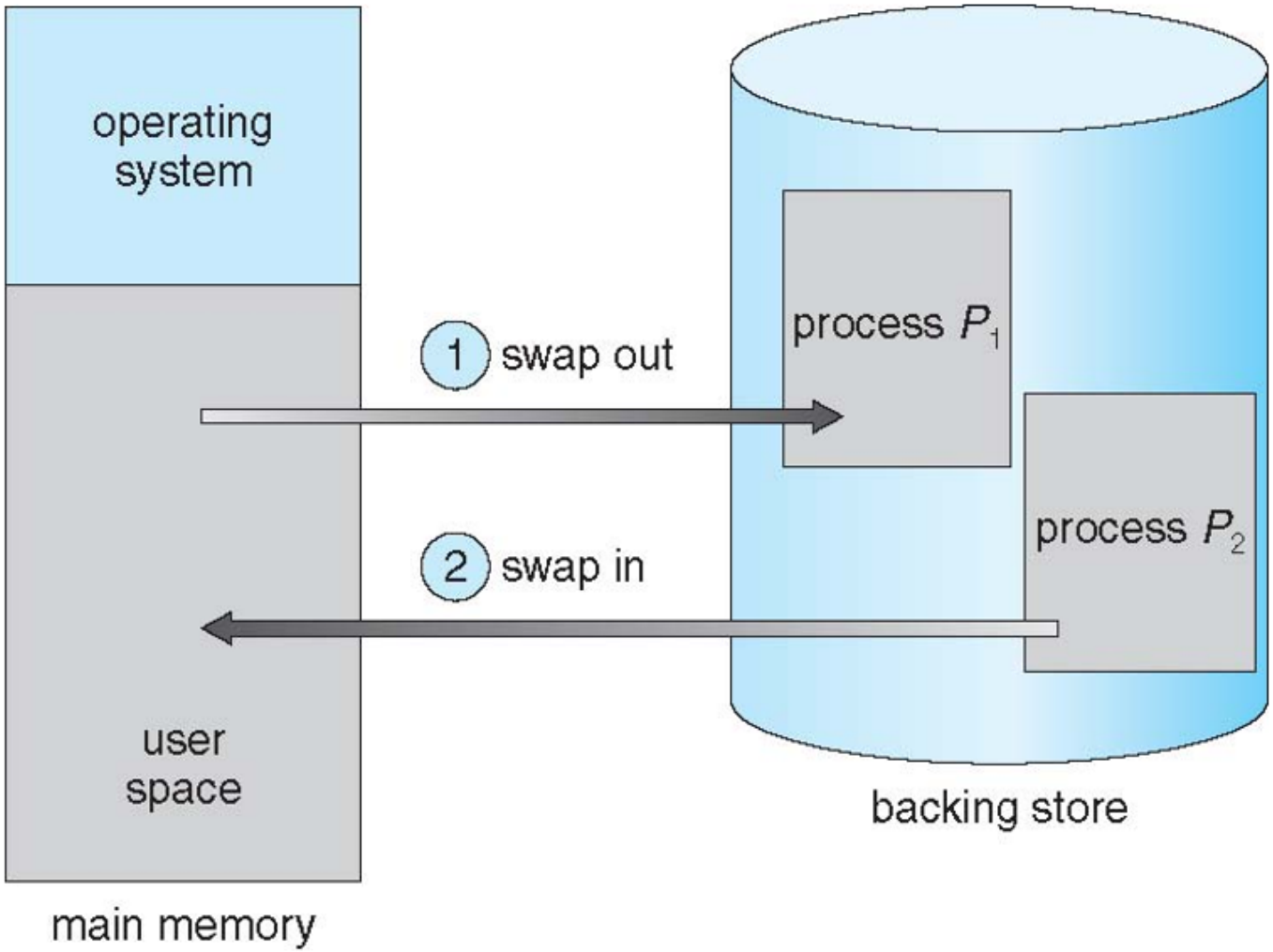
Address Binding

- Address binding of instructions and data to memory addresses can happen at three different stages:
  - **Compile time:** If the starting memory location is known beforehand, the program is compiled and an absolute code will be generated with fixed addresses.
  - **Load time:** If address binding is not done at compile time, the compiler generates a relocatable code. The loader then adjusts addresses based on where the program is placed in memory.
  - **Execution time:** Address binding occurs during program execution. The CPU generates logical (virtual) addresses, which are dynamically translated into physical addresses by the **Memory Management Unit (MMU)**.

Logical vs. Physical Address Space

- The concept of a logical address space that is bound to a separate physical address space is central to proper memory management
  - **Logical address** – generated by the CPU; also referred to as virtual address.
  - **Physical address** – address seen by the memory unit.
- **Logical address space** is the set of all logical addresses generated by a program.
- **Physical address space** is the set of all physical addresses generated by a program.

Swapping

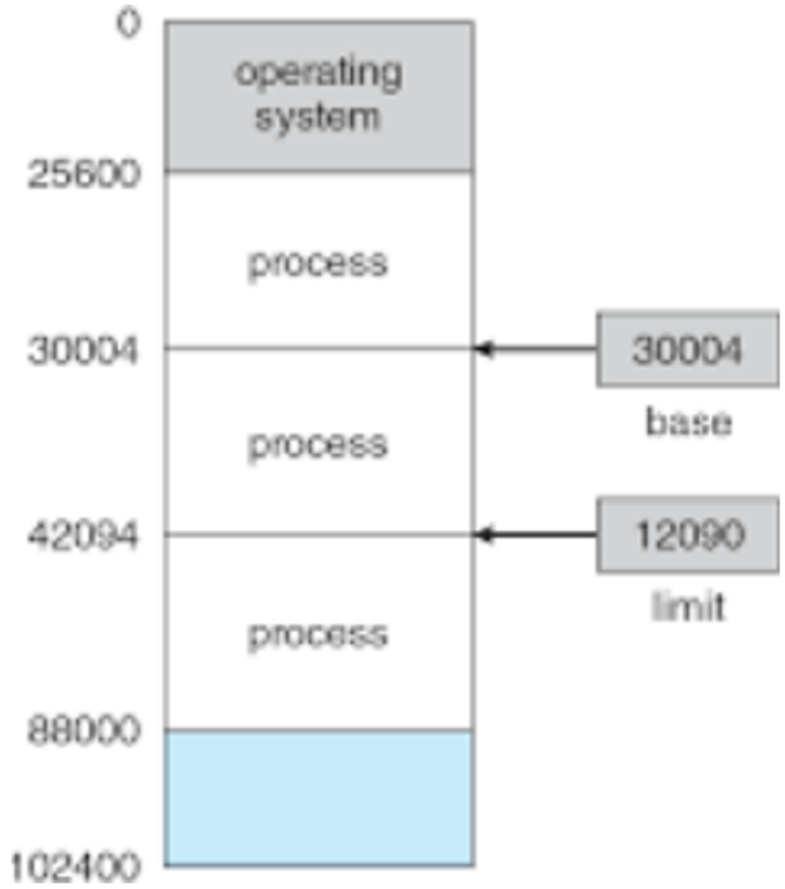

operating system
user space
main memory
1 swap out
2 swap in
process P_1
process P_2
backing store

Contiguous Allocation

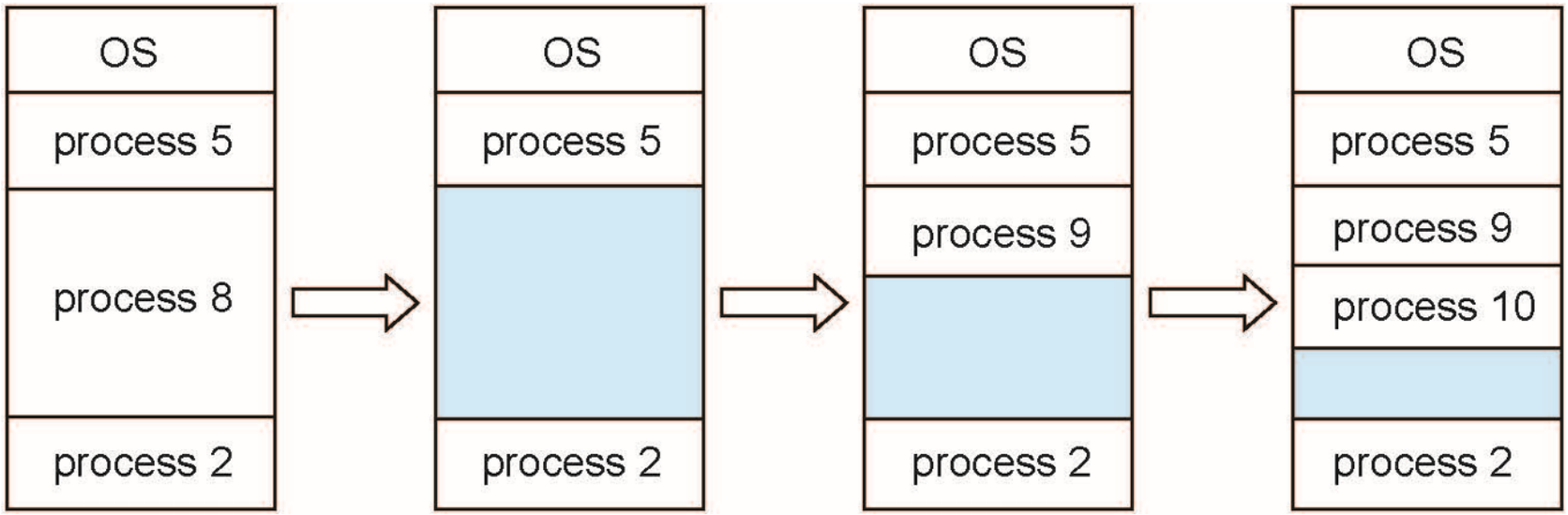
- A memory management technique in which each process is allocated a single continuous block of memory in RAM.
- When a process is loaded into memory, the OS finds a contiguous (adjacent) block large enough to hold it.

Contiguous Allocation (Cont.)

- Two registers provide address protection between processes:
  - **Base register:** smallest legal address space
  - **Limit register:** size of the legal range



Multiple-Partition Allocation



Dynamic Storage-Allocation Problem

How to satisfy a request of size n from a list of free holes?

- **First-fit:** Allocate the **first** hole that is big enough.
- **Best-fit:** Allocate the **smallest** hole that is big enough; must search entire list, unless ordered by size. Produces the smallest leftover hole.
- **Worst-fit:** Allocate the **largest** hole; must also search entire list. Produces the largest leftover hole.

First-fit and best-fit are better than worst-fit in terms of speed and storage utilization.

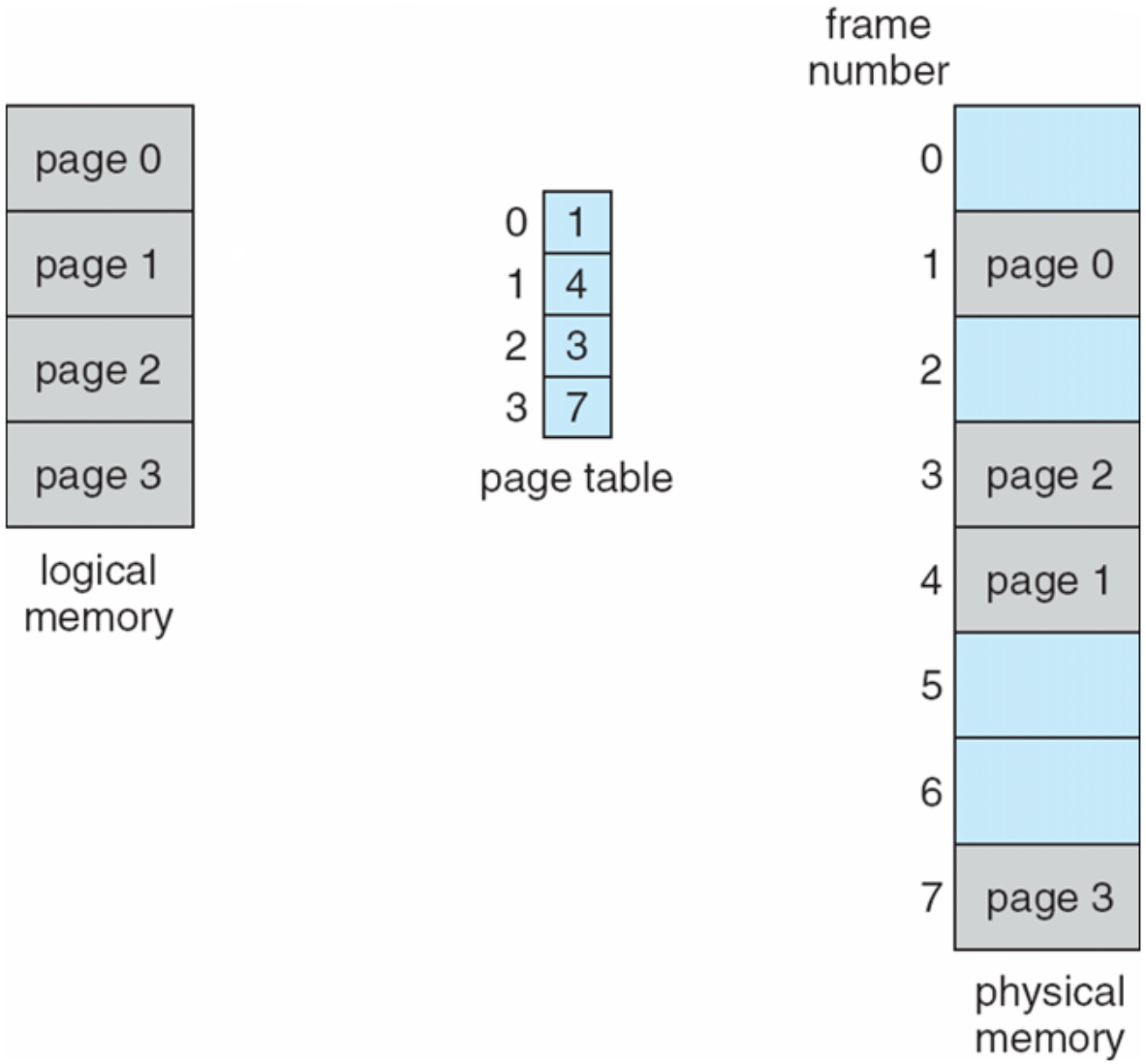
Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous.
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.

External Fragmentation and Compaction

- Reduce external fragmentation by **compaction**.
  - Shuffle memory contents to place all free memory together in one large block.
  - Compaction is possible only if relocation is dynamic, and is done at execution time.

External Fragmentation and Paging

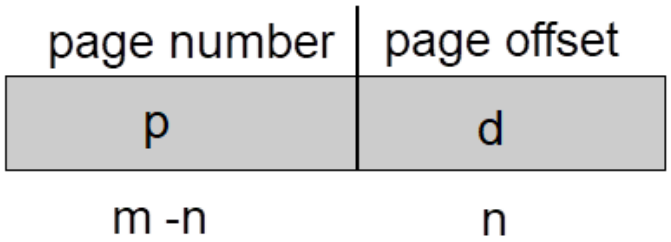


External Fragmentation and Paging (Cont.)

- Divide physical memory into fixed-sized blocks called **frames**.
- Divide logical memory into blocks of same size called **pages**.
- Keep track of all free frames.
- Set up a **page table** to translate logical to physical addresses.

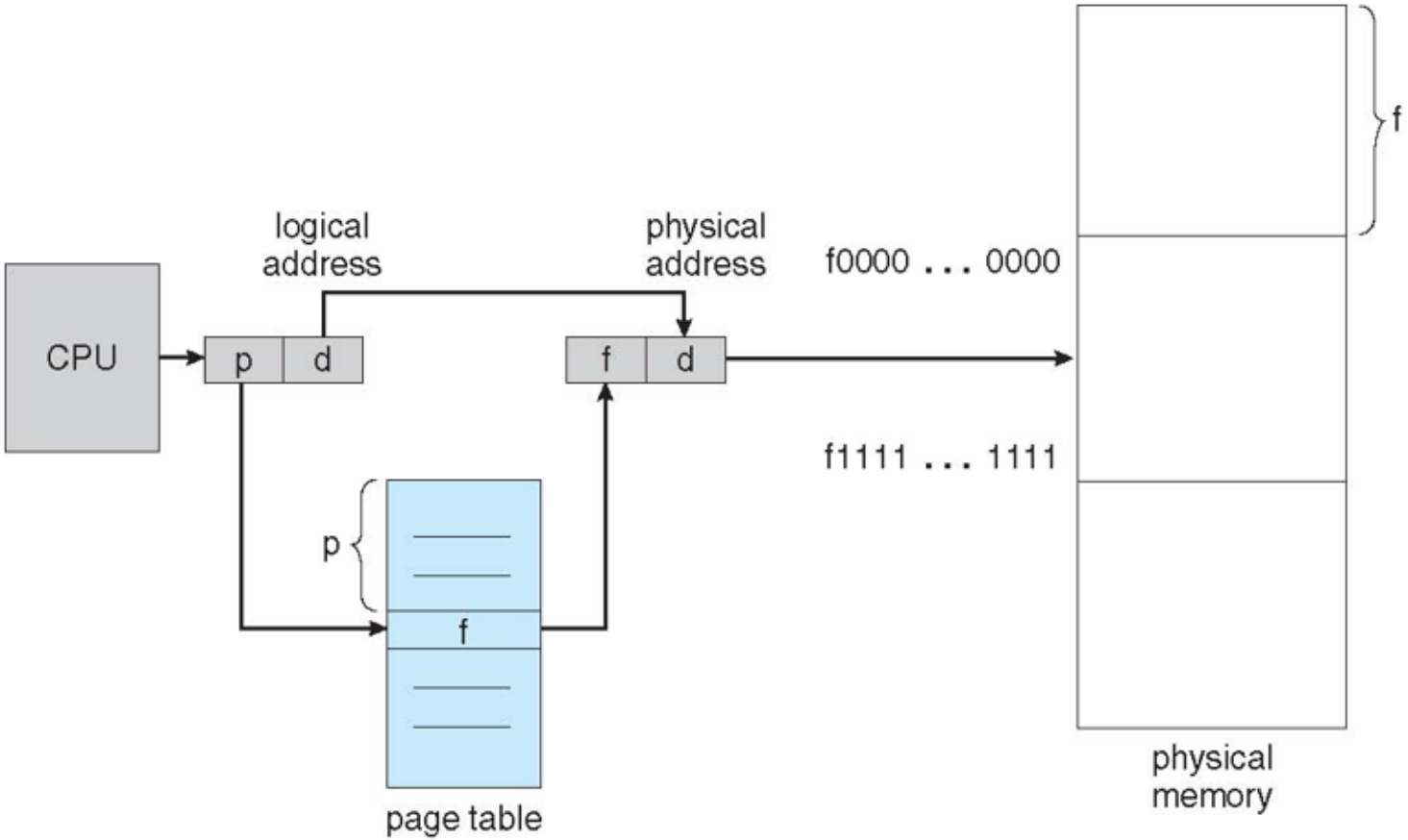
Address Translation Scheme

- Address generated by CPU is divided into:
  - **Page number (p)** – used as an index into a page table which contains base address of each page in physical memory.
  - **Page offset (d)** – combined with base address to define the physical memory address sent to the memory unit.

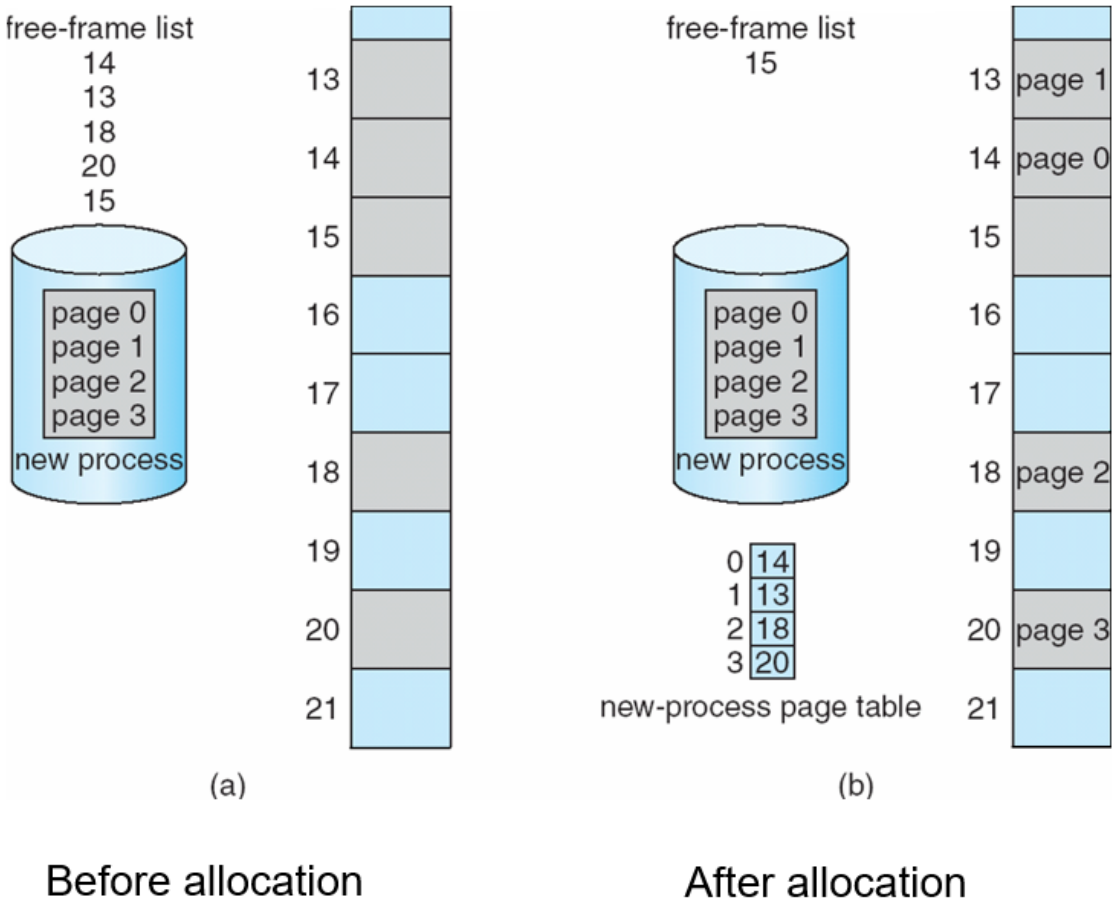


  - For given logical address space 2^m and page size 2^n.

Paging Hardware



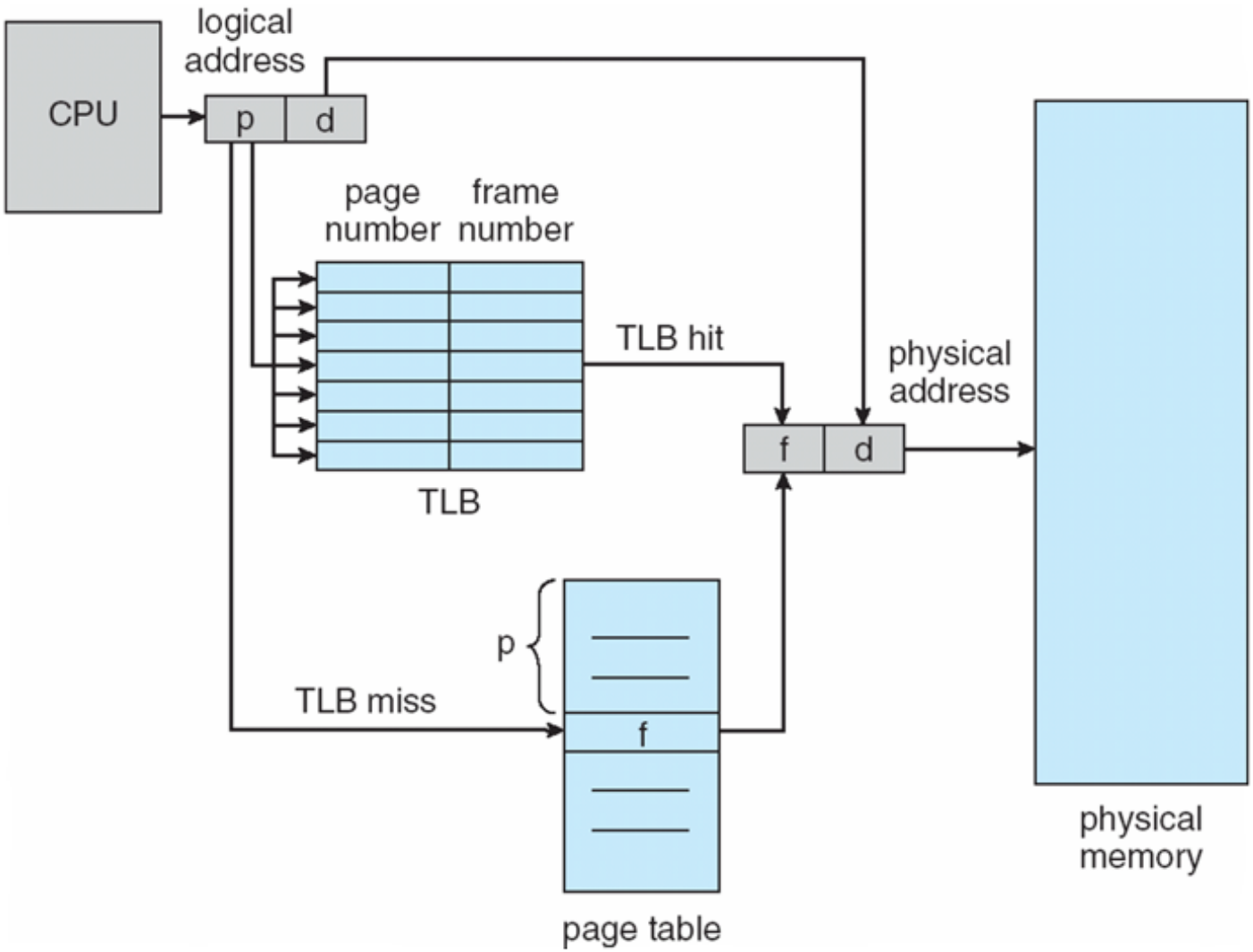
Free Frames



Before allocation

After allocation

Paging Hardware With Associative Memory (TLB)



Effective Access Time

- Associative Lookup $= \epsilon$ time unit
  - Can be $< 10\%$ of memory access time
- Hit ratio $= \alpha$
  - Hit ratio - Percentage of times that a page number is found in the associative registers

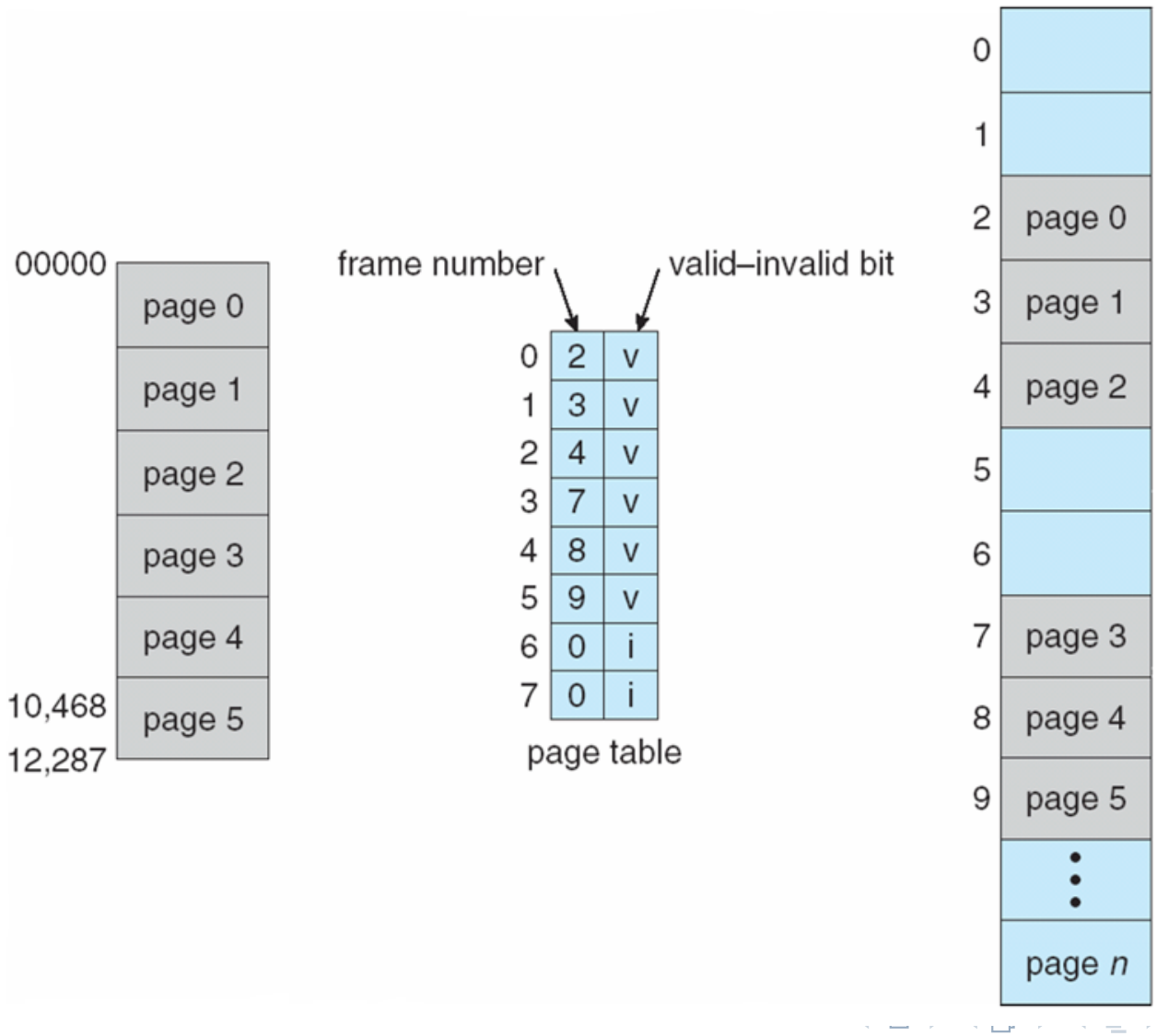
$$EAT = (ma + \varepsilon) \cdot \infty + (2ma + \varepsilon) \cdot (1 - \alpha)$$

- Consider $\alpha = 80\%$, $\epsilon = 20$ns for TLB search, 100ns for memory access: $\text{EAT} = (100 + 20)\ \text{x}\ 80\% + (200 + 20)\ \text{x}\ 20\%$

Memory Protection

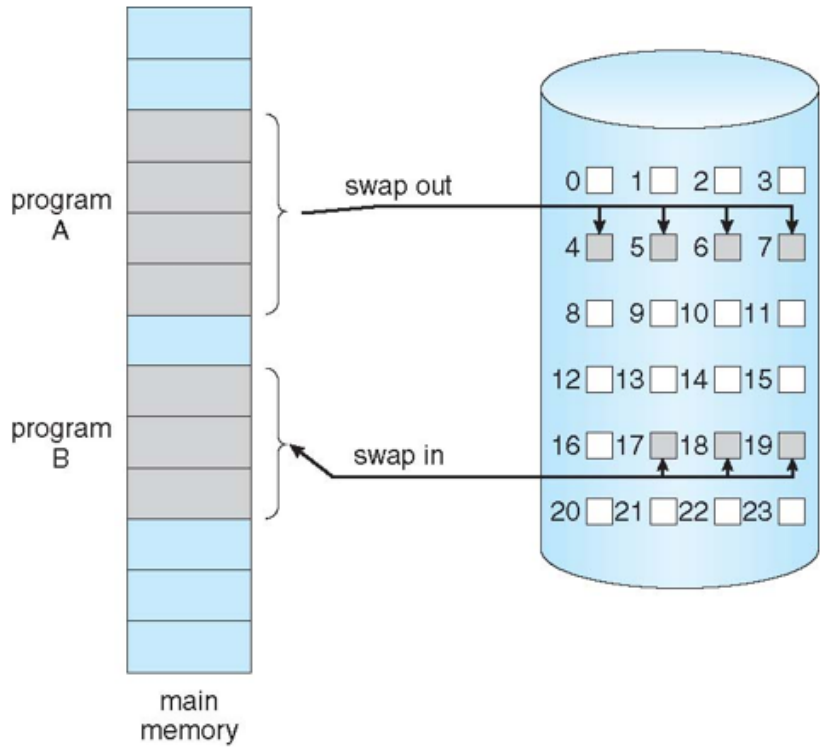
- Memory protection implemented by associating a **protection bit** with each frame to indicate if read-only or read-write access is allowed.
  - Can also add more bits to indicate page execute-only, and so on.
- **Valid-invalid bit** attached to each entry in the page table:
  - **"valid"** indicates that the associated page is in the process' logical address space, and is thus a legal page.
  - **"invalid"** indicates that the page is not in the process' logical address space.

Valid (v) or Invalid (i) Bit In A Page Table



Demand Paging

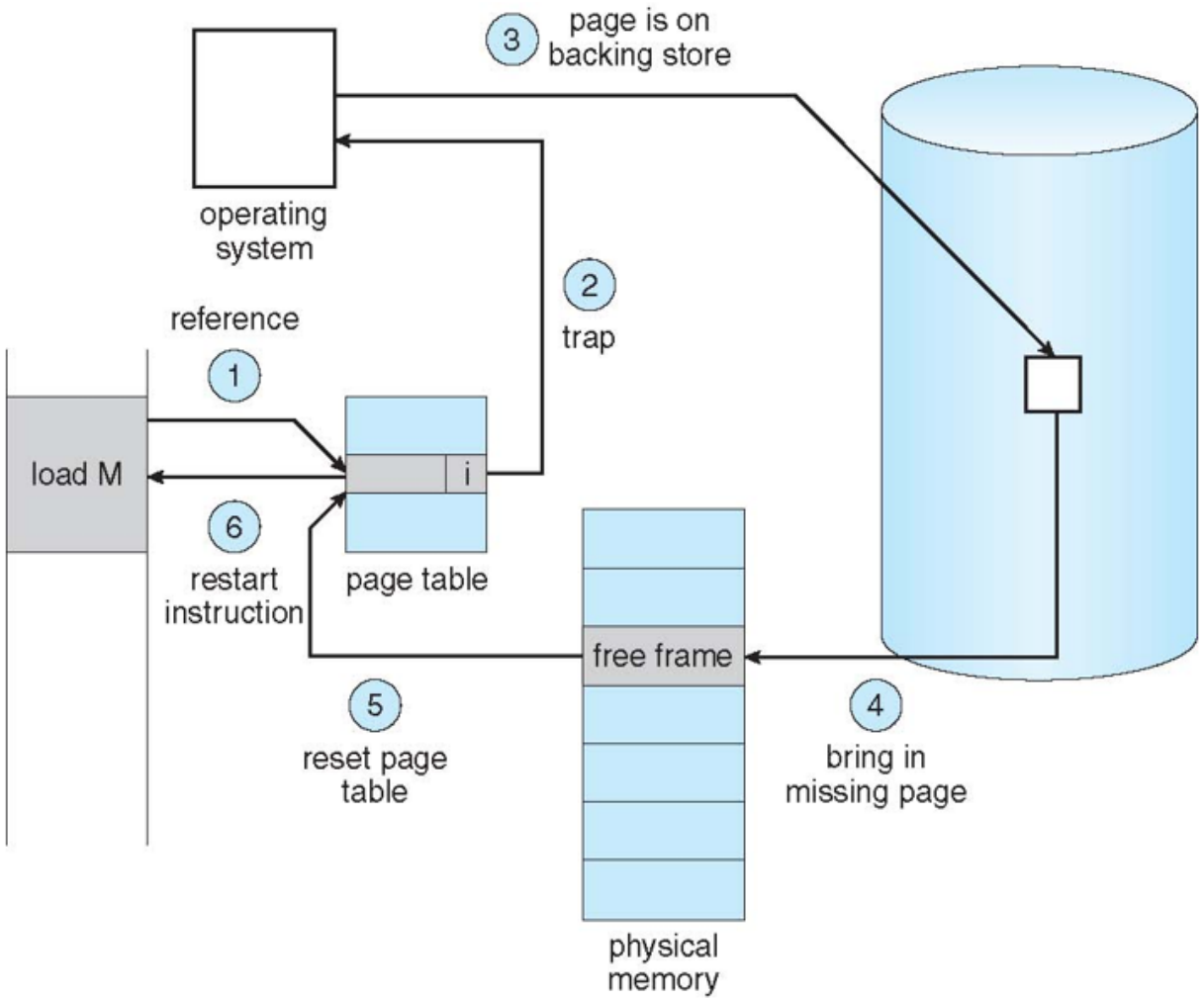
- Demand paging is a technique used in virtual memory where **pages are loaded into RAM only when they are actually needed**, rather than loading the entire program in advance.



Page Faults

- A program starts execution, but not all its pages are in memory.
- When the CPU tries to access a page that is not in RAM, a page fault occurs.
- The operating system finds the required page on the hard disk.
- Loads it into a free frame in RAM.
- The program continues execution.

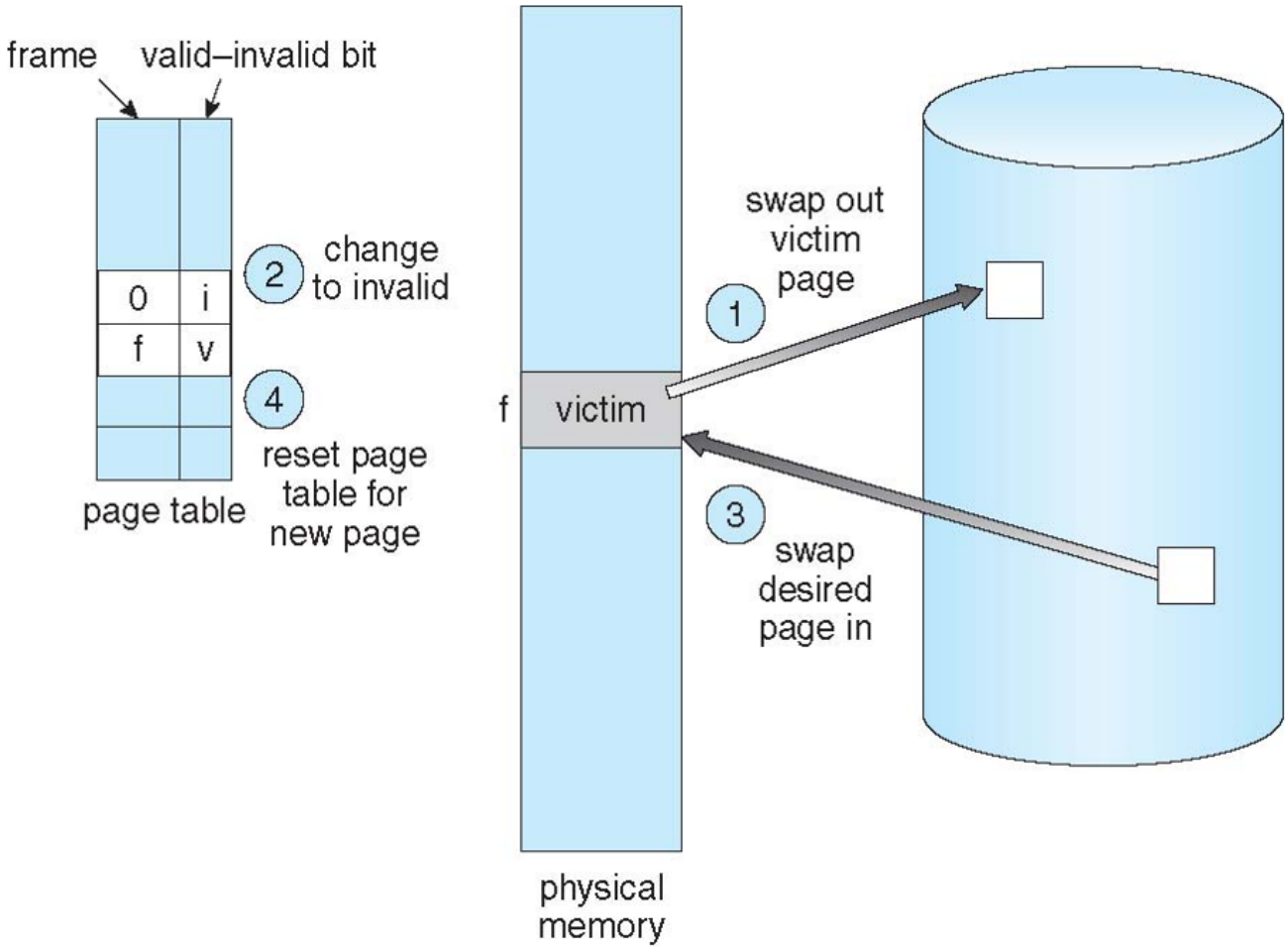
Steps in Handling a Page Fault



If No Free Frames in RAM

1. Find a free frame:
   - If there is a free frame, use it.
   - If there is no free frame, use a **page replacement algorithm** to select a **victim frame.**
   - Write victim frame to disk if **dirty.**
2. Bring the desired page into the (newly) free frame; update the page and frame tables.
3. Continue the process by restarting the instruction that caused the trap.

Page Replacement



Page-Replacement Algorithms

- There are many page replacement algorithms.
- We want an algorithm that results in the **lowest page-fault rate**.
- Evaluate algorithm by running it on a particular string of memory references (**reference string**) and compute the number of page faults on that string.

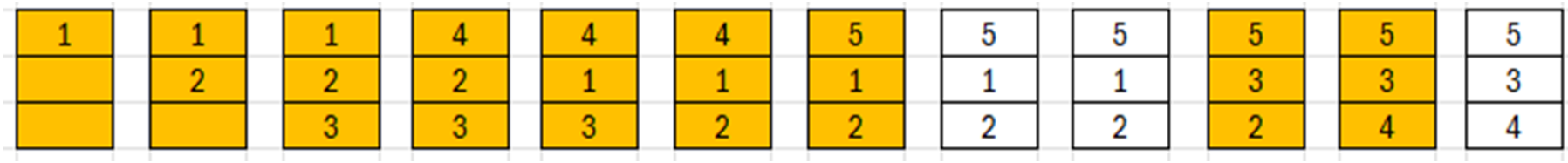
First-In-First-Out (FIFO) Algorithm

- Use FIFO queue: replace the page at the **head** of the queue, and insert a new page at the tail.
- 3 or 4 frames (3 or 4 pages can be in memory at a time per process)

- FIFO replacement suffers from **Belady's anomaly** – more frames can result in more page faults.

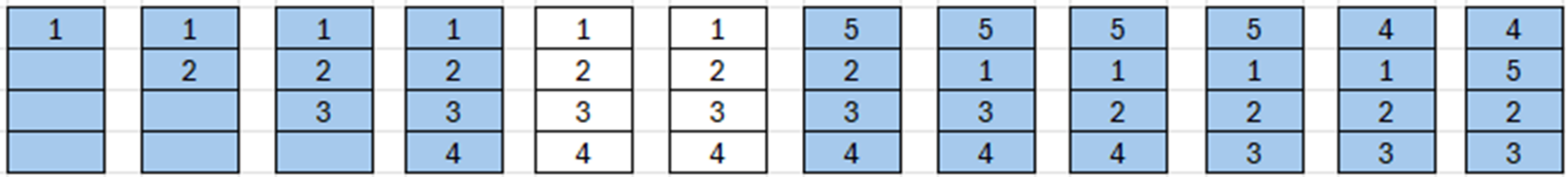
1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

If 3 Frames are in the RAM



9 page faults

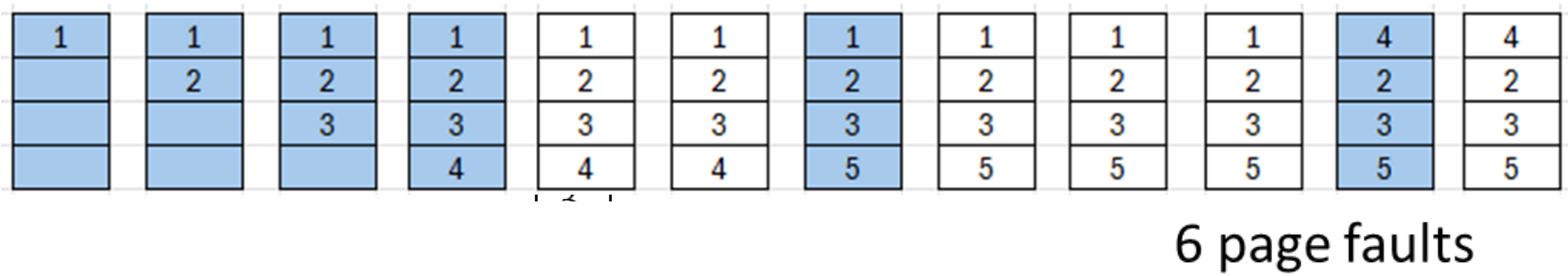
If 4 Frames are in the RAM



10 page faults

Optimal Page Replacement Algorithm (OPT)

- OPT replaces the page that will **not be used for the longest period of time**.
- 4 frames example: `1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5`

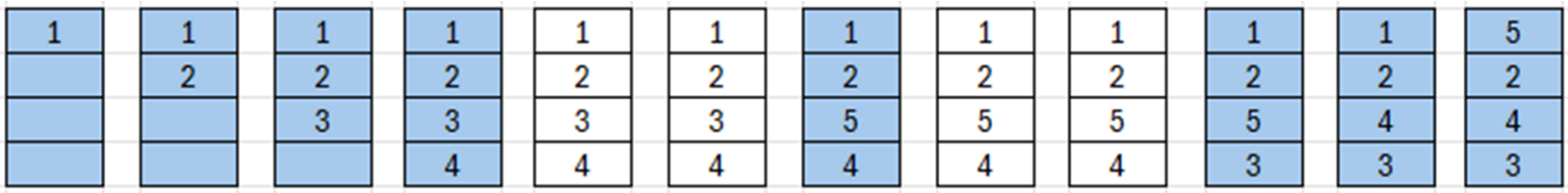


6 page faults

- For a fixed number of frames, OPT has the lowest page fault rate of all algorithms. It also never suffers from Belady's anomaly.
- Problem: It is difficult to implement – it requires future knowledge of the reference string!

Least Recently Used (LRU) Algorithm

- **LRU replaces the page that has not been used for the longest period of time**.
- Reference string: `1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5`



1	1	1	1	1	1	1	1	1	1	1	5
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	5	5	5	5	4	4
			4	4	4	4	4	4	3	3	3

8 page faults

- A class of algorithms that do **not** suffer from Belady's anomaly are called *stack algorithms*.

Page-Buffering Algorithms

- Keep a pool of free frames, always.
  - Then a frame is available when needed, not found at fault time.
  - Read page into free frame and select victim to evict and add to free pool.
  - When convenient, evict victim.
- Possibly, keep list of modified pages.
  - When backing store is otherwise idle, write pages there and set to non-dirty.

End of Lecture 11

Thank You